

Дополнительная лекция по Си № 11

26 мая 2025 г.

Коновалов А.В.

Типы данных с фиксированным размером

Стандарт не определяет размер встроенных в язык типов данных: `char`, `int`, `short int`, `long int`, `long long int` и их беззнаковые варианты. Утверждается, что размеры всех объектов (т.е. участков памяти, к которым можем обратиться) по размеру кратны `char`'у, операция `sizeof` возвращает размер объекта в размерах `char`'а. Т.е. по определению `sizeof(char)` всегда равен единице. Писать ли в программе `sizeof(char)` — исключительно вопрос стиля. Например, при выделении памяти под массив можно следовать общему принципу: всегда писать как `malloc(count * sizeof(T))`, где `T` — тип элемента, не делая исключения для `char`'ов.

Вообще, я рекомендую при выделении динамической памяти под массив писать `sizeof(*v)` или `sizeof(v[0])`, где `v` — переменная, куда массив присваивается:

```
T *p = malloc(100500 * sizeof(*p));
T *q = malloc(100500 * sizeof(q[0]));
```

Стандарт требует только то, что `short` и `int` должны включать числа от -2^{15} до $2^{15} - 1$, `long` — от -2^{31} до $2^{31} - 1$ и аналогичные ограничения для беззнаковых вариантов. Т.е. `short` и `int` должны быть как минимум 16-разрядными, `long` — 32-разрядным. А также должно выполняться ограничение

`sizeof(char) ≤ sizeof(short) ≤ sizeof(int) ≤ sizeof(long) ≤ sizeof(long long)`

Конкретные размеры оставлены на усмотрение платформы. Например, для 16-битных процессоров удобно использовать `int` как 16-разрядное число, т.к. этот тип наиболее употребительный и работа с этим типом поддерживается процессором. Так, например, было на процессоре 8086 в среде MS-DOS и ранних версиях Windows (предшествовавших Windows 95).

Тип `long long` был добавлен в стандарте C99.

На процессоре 8086 и в среде DOS `sizeof(int) = 2`, `sizeof(long) = 4`. На уже 32-битном процессоре 80386 (и последующих) и в среде Windows 95 или Windows NT `sizeof(short) = 2`, `sizeof(int) = 4`, `sizeof(long) = 4`.

Когда AMD выпустила 64-битное расширение платформы x86, то для упрощения переноса программ, написанных для 32-разрядной Windows, решили сохранить `sizeof(long) = 4`, а для представления 64-разрядных целых чисел нужно было использовать тип `long long`. Поэтому 64-разрядные компиляторы для Windows предполагают `sizeof(long) = 4`.

На 64-разрядном Linux тип `long` уже считается 64-разрядным.

Очевидно, что если Стандарт не закрепляет размер встроенных типов, писать переносимые программы, требующие целые типы фиксированного размера сложнее: приходится при помощи `typedef` описывать синонимы типов фиксированного размера, которые на разных платформах делать разными.

Поэтому в стандарте Си99 решили стандартизировать эту практику: ввести заголовочный файл `<stdint.h>`, в котором определить типы с именами: `int8_t`, `int16_t`, `int32_t`, `int64_t`, `uint8_t`, `uint16_t`, `uint32_t`, `uint64_t`. Если платформа какие-то типы не поддерживает, то они просто не будут определены: например, могут быть не определены типы `int64_t` и `uint64_t`.

К слову, был суперкомпьютер Cray (названный по имени своего создателя Сеймура Крэя), он был 64-разрядным и на нём

```
sizeof(short) = sizeof(int) = sizeof(long) = 8
```

т.е. все целочисленные типы, кроме `char`, были 64-разрядными. Это необычно, но полностью соответствует Стандарту.

Типы `intXX_t` и `uintXX_t` определены как синонимы (`typedef`) ко встроенным числовым типам, к каким конкретно — зависит от платформы.

Примечание. При объявлении целочисленных типов на Си, можно опускать `int`, а также у знаковых типов `signed`, у беззнаковых — `unsigned`. Следующие четыре описания полностью эквивалентны:

- `short`
- `short int`
- `signed short`
- `signed short int`

Причём порядок может быть любым: `int short signed` тоже будет корректным.

Все целые типы, кроме `char`, по умолчанию знаковые. `char`, в зависимости от платформы и ключей компиляции, может быть по умолчанию как знаковым, так и беззнаковым. Для случая беззнакового по умолчанию `char`'а ввели ключевое слово `signed`. Объявление `signed x`; идентично `int x`;

Функции семейства `printf` и `scanf` позволяют указывать размер аргумента при выводе: `%d` — `int`, `%ld` — `long int`, `%lld` — `long long int`, `%hd` — `short int`, `%hhd` — `char`.

Если переменная описана как `int64_t`, то какой спецификатор в `printf` использовать? Если `int64_t` синоним `long`, то `l` (`%ld`), если `long long` —

ll (%lld). Для написания переносимой программы, использующей типы из <stdint.h> и ввод-вывод функциями семейства printf и scanf, в Стандарте есть другой заголовочный файл: <inttypes.h>, содержащий макросы с именами PRI..., SCN... для функций printf и scanf соответственно. После указывается спецификатор d, x, o и затем количество разрядов: 8, 16, 32, 64.

```
int32_t x = 10;  
uint64_t y = 20;
```

```
printf("x = %" PRIu32 ", y = %" PRIu64 "\n", x, y);
```

Как мы помним, мы можем написать несколько строковых констант подряд, они неявно конкатенируются в одну:

```
const char *p = "hello" " " "world";
```

идентично

```
const char *p = "hello world";
```

Тоже самое в форматной строке printf. Если PRIu32 идентично "d", а PRIu64 идентично "llx", то следующие строки эквивалентны:

```
printf("x = %" PRIu32 ", y = %" PRIu64 "\n", x, y);  
printf("x = %" "d" ", y = %" "llx" "\n", x, y);  
printf("x = %d, y = %llx\n", x, y);
```

Также в <stdint.h> описаны верхние и нижние границы для чисел: INT8_MIN, UINT32_MAX и т.д., а также типы intmax_t и uintmax_t, означающие доступное число наибольшей разрядности. Есть и другие определения, которые мы рассматривать не будем.

Типы, зависящие от платформы

Помимо встроенных типов, типов известной разрядности, есть также синонимы типов, значение которых зависит от платформы:

- size_t (C89, <stddef.h>, <stdlib.h>, <stdio.h>, <stdint.h>) — это беззнаковый тип значения, возвращаемого операцией sizeof. Гарантируется, что размер любого непрерывного куска памяти может быть описан переменной типа size_t.
- ptrdiff_t (C89, <stddef.h>) — тип разности двух указателей, знаковый тип.
- intptr_t, uintptr_t (C99, <stdint.h>) — целые типы, размер которых не меньше размера указателя. Гарантируется, что любой указатель можно преобразовать к типу intptr_t и uintptr_t, потом преобразовать обратно и снова получить корректный указатель.

Обычно считается, что размер этих типов совпадает с размером указателя и разрядностью платформы (т.е. 32-разрядной x86 все они 32-разрядные, на 64-

разрядной — 64-разрядные), и обычно это верно. Но по смыслу типы разные, и могут быть экзотические платформы, где это не так.

В многозадачных операционных системах каждый процесс имеет собственное виртуальное адресное пространство, которое по размеру может даже превышать доступный размер физической памяти. Адреса виртуальной памяти у различных процессов как-то должны отображаться на адреса физической памяти. Сейчас используется страничная организация виртуальной памяти: память делится на страницы фиксированного размера (например, 4 Кбайта), страницы виртуальной памяти отображаются на страницы физической. Т.е. для каждого адреса виртуальной страницы указывается адрес физической (или не указывается) и права доступа к ней: возможность чтения, записи и исполнения. При обращении к странице происходит проверка её наличия и наличие прав доступа.

Исторически существовали платформы с сегментной памятью. В этом случае отображение виртуальной памяти на физическую обеспечивалось при помощи сегментов: для сегмента указывался его базовый адрес в физической памяти, размер, права доступа и некоторые другие атрибуты. Адрес в виртуальной памяти представлял собой кортеж из номера сегмента и смещение внутри него. При обращении к памяти процессор заглядывал в таблицу сегментов, находил базовый адрес сегмента, проверял, что смещение меньше длины сегмента и прочитывал значение из физической памяти по сумме базового адреса и смещения из указателя. Если в таблице сегментов сегмент отсутствовал, либо смещение выходило за границы, либо не было прав доступа, то происходила ошибка сегментирования, которая обрабатывалась операционной системой: если сегмент был выгружен в файл подкачки, то он загружался с диска, если это сегмент стека, то он мог расшириться автоматически, либо вообще процесс мог завершиться из-за ошибки сегментирования.

Сегментная память уже давно ушла в прошлое, но название сигнала, который посылается unix-подобных системах при недопустимом обращении к памяти по историческим причинам сохранился: `SEGVFAULT` или ошибка сегментирования.

На машинах с сегментной адресацией памяти невозможно выделить непрерывный кусок, размер которого превышает размер сегмента. Поэтому если на условной машине указатели 32-разрядные, делятся на 16 бит номера сегмента и 16 бит смещения, то выделить массив более 64 Кбайт будет невозможно.

На такой условной машине величина `size_t` будет 16-разрядной, а `intptr_t` и `uintptr_t` — 32-разрядными. Можно будет преобразовать указатель в `uintptr_t` и битовыми операциями выделить номер сегмента (`x >> 16`) и смещение в нём (`x & 0xFFFF`).

Стандарт языка Си должен быть применим и к машинам с сегментной памятью, и к машинам со страничной памятью, и к машинам вообще без виртуальной памяти (например, микроконтроллерам), поэтому Стандарт разрешает использовать адресную арифметику только внутри одного объекта (массива или структуры). Разность указателей определена, если два указателя

указывают на элементы одного и того же массива и равна числу элементов между ними. Также является допустимым указатель, указывающий на адрес вслед за самым последним элементом.

Вычислять разность указателей, ссылающихся на элементы разных массивов, нельзя — по Стандарту данная операция не определена. Действительно, на машинах с сегментной памятью это будет бессмысленно: или для оптимизации при вычислении разности будут учитываться только поля смещений, либо мы ещё получим разность номеров сегментов, что никакого смысла не имеет.

Макрос `offsetof` из `<stddef.h>` оставляется вам на самостоятельное осмысление.